Event Time Dilation and the Variance Clock

Two clocks, day-weighted events, and a price-preserving vol drop

Vol-Fitter Technical Notes, No. 11

Vol-Fitter

July 27, 2026

Abstract

Variance does not accrue uniformly in calendar time: a scheduled event — earnings, an FOMC print, a court ruling — packs the variance of several normal days onto one. This note documents the Vol-Fitter's *variance clock*, which makes that explicit by carrying two times: calendar time t, and a dilated variance time τ in which an event day counts as several. Each event adds a number of *extra equivalent days*; the surface is fitted and quoted in weighted years $\tau = \tau_{\text{days}}/365$. The key invariant is price-preservation: total variance $w(k)$ is fixed by the observed option price and is clock-independent, so the working implied volatility $\sqrt{w/\tau}$ *drops* when an event is inserted before the expiry — exactly the overnight-vol-crush traders expect. An optional normalization rescales all days so the one-year variance budget is unchanged, making events *redistribute* variance within the year rather than add it. The calendar can be typed in or *auto-calibrated*: a solver places events so the event-time forward variance is as flat as possible, inferring the calendar from the term structure's kinks, after which it is editable. Below one day the clock continues: an opt-in *intraday session clock* values each node from the snapshot timestamp to the expiry's exact settlement instant and splits each trading day's variance between session and overnight — at its default parameters it nests the legacy day convention exactly, and the research levers make a live 0DTE's clock "remaining trading minutes". With no events the variance clock collapses to calendar time and every downstream fit is byte-identical to the plain pipeline. A worked example shows a 4-extra-day earnings event lifting short-dated implied vol and decaying away, with a working-vol drop of 36 vol bps at a fixed expiry just past the event.

Contents

1 Why two clocks

A diffusion model assumes variance accrues at a steady rate: total variance $w = \sigma^2 t$ is linear in calendar time. Real markets violate this around scheduled events, where a single day carries the variance of many. Forcing such a surface onto a single calendar clock produces the familiar pathology — a short-dated implied vol that looks "too high" and a term structure with a kink that a smooth model cannot represent without distorting the smile.

Heuristic

Separate the *calendar* (how many days until expiry) from the *variance budget* (how much variance those days carry). An earnings day is one calendar day but, say, five days' worth of variance. If we measure time in *variance days*, the diffusion looks steady again — and the "too high" short-dated implied vol is just the same total variance divided by a smaller calendar time. The event clock is the change of variable that restores $w = \sigma^2 \tau$ with a constant σ.

Invariant

1. **Price preservation.** The clock never touches $w(k)$; it changes only the volatility read-off $\sqrt{w/\tau}$. Adding an event cannot change an option price, hence cannot introduce arbitrage.
2. With an empty calendar, $\tau = t$ exactly and the entire downstream pipeline is byte-identical.
3. Every consumer — every fit, the LV surface, the term structure, the table — reads the *same* τ; there is one variance clock per node, not one per view.
4. Editing the calendar refits only the affected ticker (per-ticker events version).
5. Below one day: the intraday session clock is off by default (byte-identical), and even *on*, its default parameters nest the legacy integer-day convention exactly (section 4).

2 The variance clock

2.1 Day-weighted variance time

Each event carries a number N_e of *extra* equivalent days (an earnings day with $N_e = 4$ counts as 5 normal days of variance). The variance time to a calendar maturity t is

$$\tau_{\text{days}}(t) = t \cdot 365 + \sum_{t_e \le t} N_e, \qquad \tau(t) = \frac{\tau_{\text{days}}(t)}{365}, \tag{1}$$

counting only events at or before t. Figure 1 plots $\tau(t)$: it tracks the calendar line until the event, jumps by $N_e/365$ there, and runs parallel afterward. The fits and quotes live on τ; with no events, $\tau(t) = t$ exactly. The clock (1), with the optional 1Y-budget normalization, is direct (matches production to 10^{-12}):

Listing 1: The event variance clock $\tau(t)$ (1) (distilled from `calib/weighted_time.py::weighted_variance_years`).

```
def weighted_variance_years(t_cal, events, normalize=False, dpy=365.0)
    :
```

```
    if t_cal <= 0: return t_cal           # events: (t_e, N_e) pairs,
        N_e = extra days
    tau_days = t_cal*dpy + sum(N for te, N in events if te <= t_cal
        and N > 0)
    if normalize:                         # rescale so a 1Y budget
        stays dpy days
         e1 = sum(N for te, N in events if te <= 1.0 and N > 0)
         tau_days *= dpy/(dpy + e1)
    return tau_days/dpy                   # tau(t); equals t_cal with
        no events
```

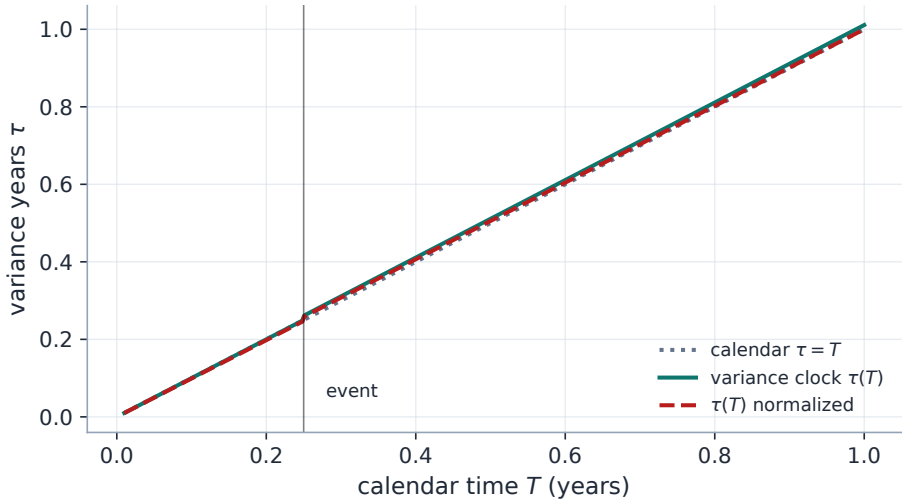

Figure 1: Variance time $\tau(t)$ for an event at $t_e = 0.25$ worth 4 extra days: it jumps at the event and runs parallel to calendar time thereafter. The normalized clock (dashed) rescales so the one-year budget is unchanged.

2.2 Price preservation

The crucial invariant is that the *total variance* $w(k)$ is fixed by the observed option price — it is the Black total variance that reprices the quote, and that is clock-independent. The variance clock only changes how we *report* it as a volatility:

$$\sigma_{\text{work}}(k) = \sqrt{\frac{w(k)}{\tau}}. \tag{2}$$

Inserting an event before the expiry raises τ at fixed w, so the working implied vol *drops*. This is the vol crush: the price is unchanged, but the *diffusive* volatility implied by it is lower once the event's variance is accounted for separately. Figure 2 shows the term-structure consequence: with the event, short-dated calendar implied vol is lifted (the event's variance compressed into few calendar days) and decays as the event becomes a smaller fraction of the maturity; without it, the term structure is flat.

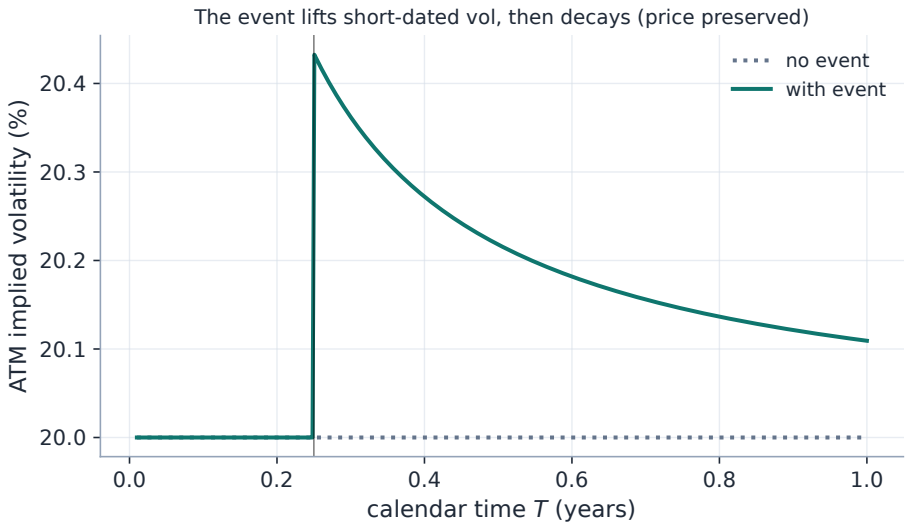

Figure 2: ATM implied-vol term structure with and without the event. The event lifts short-dated vol and decays away; the option *prices* are preserved, only the implied-vol reading changes.

2.3 Normalization

By default the cumulative weight exceeds the calendar-day count ($\tau > t$). With normalization on (an Options toggle), all days — event days included — are rescaled by a single factor so the one-year weight budget stays 365:

$$\tau_{\text{days}}(t) \;\longmapsto\; \tau_{\text{days}}(t) \cdot \frac{365}{365 + \sum_{t_e \le 1} N_e}. \tag{3}$$

Then the one-year variance time — and the one-year implied vol — are identical to a no-event year: events only *redistribute* variance within the year, they do not add it. Which convention to use is a desk choice: un-normalized treats the events as genuinely extra variance, normalized treats them as a known reallocation of a fixed annual budget.

3 How it threads the pipeline

The dual clock is prepared once per node by the clock of listing 1: τ is stored as `prepared.tau` beside the calendar `prepared.t`, and every fit (parametric and local-vol), the local-vol grid, the term structure and the option table consume τ in place of t.

Remark 1 (One production clock, one legacy clock, one unit trap)**.** Two dilation clocks exist in the codebase and must not be conflated. The production clock is `calib/weighted_time.py` — day-weighted, exactly (1), the one every fit consumes. The older `calib/event_time.py` (`EventClock`) survives only for term-structure interpolation. And the unit trap: `weight` is *extra equivalent days*, never years — confirmed both by the call site (the pairs feed (1) directly) and by the auto-calibrator, which constructs `EventSpec(weight=days)`. The `EventSpec` schema docstring said *years* until 2026-07-09 (now fixed to the day semantics documented here); the legacy clock's comments remain the stale party.

Total variance is interpolated *linearly in* τ between quoted nodes — flat forward variance per weighted-time unit — so the dense vol curve $\sqrt{w/\tau}$ stays bounded near zero and sensible past the

last expiry. The term-structure view uses the shared calendar (not the per-node events) so the displayed curve is coherent across expiries. Changing the event calendar bumps the per-ticker events version, forcing a refit of only that name; with an empty calendar the whole pipeline is byte-identical to calendar time.

Nor must the calendar be hand-entered. The Term workspace's *auto-calibrate* action infers it from the ATM term structure: placing one candidate event before each expiry up to a chosen horizon (`maxExpiry`), it solves for the non-negative extra days N_e that make the *event-time* forward variance between consecutive expiries, $(w_i - w_{i-1})/(\tau_i - \tau_{i-1})$, as flat and as monotone-increasing as possible while keeping the events small and sparse (a bounded L-BFGS-B solve; events under half a day are thresholded away, so the result is a short list). The calendar-time forward variance $(w_i - w_{i-1})/(t_i - t_{i-1})$ is event-*invariant* — prices fix w — so the objective works on the dilated clock, the one events actually move: since an event can only *lengthen* an interval's weighted time, it pulls *down* the forward-variance spike an earnings-packed interval produces, toward its neighbours. The calendar is thus read off exactly the kinks the clock was built to explain, installed as the shared per-ticker calendar, and edited like any hand-entered one (`calib/event_autocalib.py`, `api/event_autocalib.py`).

4 Below one day: the intraday session clock

The day-granular clock is meaningless at hours to expiry: a 0DTE's remaining life is set by the wall clock and by *how* variance accrues through a trading day. The opt-in intraday clock (`intradayClock`, off by default and byte-identical off; `calib/intraday_time.py`) extends the day-weight convention below one day in two moves. First, *valuation instants become exact*: each node is valued from the chain snapshot's timestamp to the expiry's *settlement instant* — the schema-v7 settlement map, with NYSE session rules (holidays, half-days, DST-correct ET boundaries) as the fallback — so a same-day expiry prices at $t = 3.5$ hours rather than the unrepresentable zero (certification case `0dte_exit_gates`). Second, *within* a trading day the day's weight is split piecewise-uniformly: a fraction `sessionVarShare` of the day's variance accrues during the exchange session (half-day sessions scale it), the remainder overnight, and a non-trading calendar day carries `nonTradingWeight` — the weekend-effect research lever.

The defaults are chosen for a nesting property, not a view: `sessionVarShare` $= 6.5/24$ is the *flat-density* share, and with `nonTradingWeight` $= 1$ any close-to-close span of N calendar days integrates to exactly N day-weights — the legacy convention, recovered to the day. The research values are where the physics lives: shares of ~ 0.7–0.9 concentrate variance in trading hours, making a live 0DTE's clock "remaining trading minutes" with a cheap overnight and a cheap weekend. The evidence for wanting them is measured, not assumed: on the 2026-07 intraday campaign (936 filter measurements, Note 15's session clock — the same physics consumed by the observation filter), 30-minute steps moved ATM vol by 19.5 bp while one overnight and a *whole weekend* both moved it ≈ 55 bp — no calendar-proportional clock can calibrate all three at once. Two scope notes: the in-session profile is uniform in v1 (the U-shaped open/close seasonality is a documented follow-up, deliberately not a knob yet), and scheduled *intraday* events need no new machinery — `EventSpec` times are year fractions, so once the clock is sub-day an 08:30 CPI print is simply an event whose fractional time sits mid-morning, composing with (1) unchanged.

5 What is original here

Event time changes are not new in spirit (Bergomi and others dilate time around events), but the implementation here has two clean properties worth singling out. First, *price preservation by construction*: the event clock never touches $w(k)$, only the volatility read-off (2), so adding an event can never change an option price — it cannot introduce arbitrage. Second, the *day-weighted* parametrization (1) makes the event size a single intuitive number (extra equivalent days) that a desk can quote and reason about, with an optional budget normalization (3) that cleanly separates "extra variance" from "reallocated variance". And the byte-identical fallback when the calendar is empty means the feature is strictly additive. The payoff is comparability: pre- and post-earnings surfaces line up across names and dates because event-time vol is the signal — calendar-time vol is the artifact.

6 Traceability

Table 1: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
No events: identity clock, byte-identical pipeline	invariant 2	`calib/weighted_time.py`	`test_weighted_time.py::test_no_events_is_identity`, `::test_no_event_byte_identical`
Event adds day weights before its date only	(1)	`calib/weighted_time.py`	`test_weighted_time.py::test_event_before_adds_day_weights`
Price preservation: IV drops by $\sqrt{t/\tau}$	(2)	`api/service.py`, `api/displayed.py`	`test_weighted_time.py::test_event_lowers_iv_by_sqrt_t_over_tau`, `::test_localvol_iv_drops_with_event`
Normalization pins the one-year budget	(3)	`calib/weighted_time.py`	`test_weighted_time.py::test_normalization_pins_one_year`, `::test_normalization_keeps_one_year_vol`
Surface mesh shares the smile's clock	section 3	`api/affine_views.py`	`test_surface_clock.py::test_surface_mesh_uses_variance_clock_like_the_smile`
Legacy clock confined to term interpolation	remark	`calib/event_time.py`	`test_event_time.py::test_interpolation_lumps_event_variance`
Auto-calibrate flattens event-time forward variance; flat input stays eventless	section 3	`calib/event_autocalib.py`, `api/event_autocalib.py`	`test_event_autocalib.py::test_spike_is_flattened`, `::test_flat_input_stays_eventless`

Claim	Object	Code anchor	Test anchor
Intraday clock nests the legacy convention at defaults; sub-day settlement valuation (cert. `0dte_exit_gates`)	section 4	`calib/intraday_time.py`, `data/expiry_time.py`	`test_intraday_time.py`, `test_intraday_0dte.py`

A Hyperparameter atlas

Table 2: Event-clock hyperparameters.

Knob	Default	Role
Surfaced		
`events`	empty	List of `EventSpec`(t_e, N_e, label); N_e = extra equivalent days. Persisted *per ticker* (its own endpoint and version), not in the global options.
`normalizeEvents`	off	Rescale all days to a fixed 365 one-year budget (3).
`maxExpiry`	—	Auto-calibrate horizon: one candidate event per expiry at or before it (section 3).
`intradayClock`	off	Sub-day settlement-instant valuation + session-weighted intraday accrual (section 4); byte-identical off.
`sessionVarShare`	6.5/24	In-session share of a trading day's variance; the default is the flat-density value that nests the legacy day convention.
`nonTradingWeight`	1.0	Day-weight of a non-trading calendar day (the weekend-effect lever); 1.0 keeps the legacy three-full-days weekend.
Hidden		
`DAYS_PER_YEAR`	365	Calendar-to-variance day conversion.
`mono_weight`	1.0	Auto-calibrate: penalty on decreasing event-time forward variance.
`sparse_weight` / `ridge_weight`	10^{-3} / 10^{-4}	Auto-calibrate: keep solved events small and sparse.
`min_event_days`	0.5	Auto-calibrate sparsity threshold: smaller solved events are dropped.

Performance. The clock is a scalar sum per node, negligible. With no events $\tau = t$ and the entire downstream pipeline is byte-identical, so the feature adds zero cost when unused; changing the event calendar refits only the affected ticker (per-ticker events version).

References

[1] L. Bergomi. *Stochastic Volatility Modeling*. CRC Press, 2016.

[2] J. Gatheral. *The Volatility Surface*. Wiley, 2006.